2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C) | 979-8-3503-2498-3/23/$31.00 ©2023 IEEE | DOI: 10.1109/MODELS-C59198.2023.00113

Mastering Reference Architectures with Modeling Assistants

Amleto Di Salle
dept. of Human Sciences
European University of Rome
Rome, Italy
amleto.disalle@unier.it

Ludovico Iovino
Computer Science area
Gran Sasso Science Institute
L'Aquila, Italy
ludovico.iovino@gssi.it

Leonardo Mariani
dept. of Informatics, Systems and Communication
Università degli Studi di Milano-Bicocca
Milan, Italy
leonardo.mariani@unimib.it

***Abstract*—Reference architectures (RA) can significantly simplify and speed up the design and evolution of software systems in virtually any domain. Rather than starting from scratch or trying to reuse models not meant for reuse, software architects can exploit RAs to initiate their designs from rigorously-defined architectural models specifically defined to support reuse. RAs can guarantee the satisfaction of key functional and non-functional requirements, including those related to technical and societal challenges, such as security and sustainability, significantly reducing the design effort and improving the design quality. The growing popularity of RAs is confirmed by the many providers, including Amazon AWS, Microsoft Azure, IBM, and Salesforce, that offer RAs for application deployment on their infrastructures. Unfortunately, RAs are expensive to define since they must comprehensively synthesize the design knowledge of specific application domains (e.g., e-commerce, automotive, etc.) and technical contexts (e.g., cloud systems, mobile apps, IoT, etc.). This challenge limits the availability of RAs to a few domains and contexts, hindering their adoption. We envision the opportunity to facilitate RA-based design by introducing a novel combination of *mining in-the-wild* and *continuous knowledge refinement* through a framework named ASSISTRA. Mining in-the-wild consists of *bots* crawling architectural models at scale (e.g., from public repos) and analyzing them continuously to automatically synthesize RAs and extract RAs-related practices that shall feed knowledge bases. Continuous knowledge refinement consists of a population of *recommender systems* that, while interactively supporting the architects in their activity using the shared knowledge bases, shall continuously refine the mined content in the knowledge base (e.g., based on architects accepting, modifying, or rejecting suggestions). This process shall generate high-quality recommendations, drastically simplifying the design activity. This paper elaborates on this vision, highlights the key challenges, and calls for novel research contributions.**

***Index Terms*—reference architecture, architectural model, modeling assistants, mining architectures**

I. Introduction

A reference architecture (RA) is "a generic architecture that provides guidelines and options for making decisions in the development of more specific architectures and the implementation of solutions" [1]. A Gartner's report claims that RAs: *"reduce the complexity of hardware and software architecture by systematically reducing environmental diversity, enables greatly increased speed and reduced operational expenses as well as quality improvements due to lower complexity, greater investment, and greater reuse"*. The need for reference architectures also became tangible in the industry due to the fast-growing number of initiatives working toward standardized architectures [2]. Practically, RAs inspire architects when designing concrete architectures [3], and they help to guarantee compliance with architectural decisions, regulatory requirements, as well as architectural qualities. For the reasons mentioned above, a RA can be defined as a general architecture that is used as a foundation for the design of concrete architectures within a given context or application domain, e.g., automotive [4]–[6], banking, the internet of things, avionics [7], and robotics [8].

RAs guide and constrain the instantiation of multiple architectures and solutions. Since it is a structural description, RAs usually do not identify the internal mechanisms that drive information processing among the components, like behavioral diagrams. An example of RA for web browsers appeared first by Grosskurth and Godfrey [9], but important industrial actors also started to catalog RAs for the benefit of the architects and developers. For instance, Amazon advertises the suggested RAs [10], and so does Microsoft for Azure [11], and also IBM [12] to assure compliance with their technologies and standard when used as templates and guidelines. An implicit relation co-exists between the implemented (concrete) architecture and its RA, meaning that, e.g., a concrete component A implements the component B of the RA[1].

Unfortunately, RAs are expensive to define since they must comprehensively synthesize and combine the design knowledge of specific application domains (e.g., e-commerce, automotive, etc.) and technical contexts (e.g., cloud systems, mobile apps, IoT, etc.). Although the knowledge necessary to specify RAs is readily available, implicitly codified in the many modeled architectures that architects have defined for their systems, it is an expensive and laborious process to integrate such knowledge into a usable RA. This challenge limits the availability of RAs to a few domains and contexts, hindering their adoption.

To address this gap, we envision the possibility of combining two key ingredients: mining in-the-wild and continuous knowledge refinement. *Mining in-the-wild* consists of exploiting the knowledge readily available in the form of architectural

[1]We simplified the relationship because it can also be an n-m, 1-n relation

979-8-3503-2498-3/23/$31.00 ©2023 IEEE
DOI 10.1109/MODELS-C59198.2023.00113

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on September 22,2025 at 12:48:27 UTC from IEEE Xplore. Restrictions apply.

models to synthesize RAs automatically. This can be achieved with bots configured to systematically navigate target spaces to extract architectural models. Many spaces are eligible as target spaces. For instance, bots could crawl the Web or search within open source repositories (e.g., GitHub), but companies could also configure them to search within their private spaces. In addition to accumulating models, the bots will regularly analyze models to produce more domain-specific RAs that architects can use to design their systems. Specific good and bad design practices (e.g., in the form of patterns or smells) could also be extracted from the accumulated models and shared with architects. The mined RAs and practices are stored in a shared knowledge base.

Continuous knowledge refinement consists of an ecosystem of Recommender Systems that share the same knowledge base and act within the architects' environments to support them during their design tasks. Recommender systems shall recommend design decisions based on the mined RAs and practices. The actions taken by architects will be exploited to update the shared knowledge so that the crowd's wisdom (of architects) will iteratively improve the mined knowledge. The vision is that the initially mined and less accurate knowledge base may evolve into a shared and more sophisticated knowledge base that can dramatically improve the productivity of architects.

To the best of our knowledge, there are no approaches to assist the architect in creating and evolving RAs and related artifacts. In this paper, we elaborate our vision and describe ASSISTRA, a framework that implements it, and we discuss the use cases that could be addressed.

II. REFERENCE ARCHITECTURES

A Reference Architecture (RA) provides methodologies and practices, usually formalized as templates, that are based on the generalization of a set of solutions for a particular category of systems or application domain. Among the various definitions of RAs proposed in the last years [13], [14], the Rational Unified Process (RUP) [15] defines an RA as "a predefined set of architectural patterns, proven for use within specific business and technical contexts, together with artifacts enabling their use". Apart from the foundational aspect, most of the existing works in literature define RAs for specific domains, e.g., automotive [5], [6], [16], autonomous vehicles [17], or IoT [18], [19]. For instance, Fig. 1 reports an example of RA for IoT [19], [20] consisting of five layers. The bottom layer is composed of `Sensors` and `Actuator` components. A sensor is a unit of hardware that reacts to a physical stimulus, such as heat, light, sound, or a specific motion, to gather information about the physical environment. An actuator is a piece of hardware that affects the physical environment. Actuators translate the instructions sent by a linked device into physical actions, converting them into electrical impulses. A `Device` is the entry point into the digital world from the physical one, and connects to actuators and/or sensors through wire, wireless, or both. A `Driver` consists of a processor and storage space that allows running software on the device to access various sensors and actuators. When a device cannot connect to other systems directly, it is linked via a gateway. A `Gateway` allows communication across heterogeneous protocols, communication technologies, and payload formats. The `IoT Integration Middleware (IoTIM)` acts as an integration layer for sensors, actuators, devices, and applications. It receives data from connected devices, processes them, and sent to `Applications`. In the following, we describe two architectures reported in [19], one open-source named FIWARE [21] and one from industry, the well-known AWS IoT [22], and we highlight that both follow the RA defined in Fig. 1. In these architectures, We use the same colors used in Fig. 1 to show how the components map to the reference architecture's components.

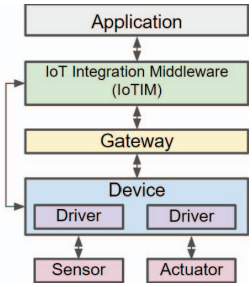

Fig. 1: Reference Architecture for IoT proposed in [19], [20]

Fig. 2 shows the FIWARE architecture and mapping onto IoT RA. First, FIWARE differentiates between `Devices` and `Next-generation service interfaces (NGSI) devices`, which integrate sensors and actuators. Thus, the device and Sensor&Actuator RA components map both of them. A device directly communicates with the `IoT Back-End` or through a `Gateway` arranged within the `IoT Edge`. The same happens for an NGSI device using `IoT NGSI Gateway`. Therefore, the IoT Edge defines the Gateway of the IoT RA. The core functionality of FIWARE is provided by the IoT Back-End and the `Data Context Broker`, which is enclosed by the IoT RA Integration Middleware.

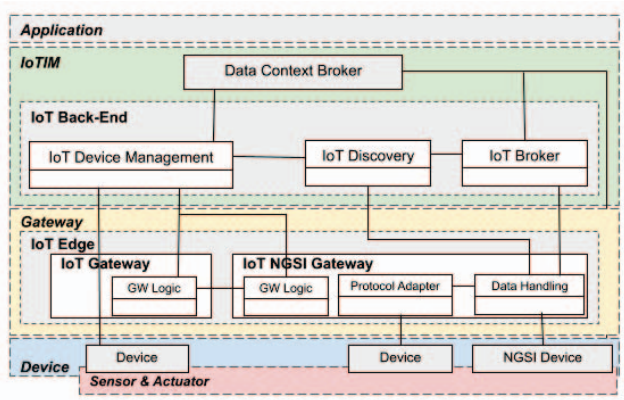

Fig. 2: FIWARE architecture for IoT

Fig. 3 depicts the AWS IoT cloud architecture and mapping

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on September 22,2025 at 12:48:27 UTC from IEEE Xplore. Restrictions apply.

onto the IoT RA. In AWS, sensors and actuators are integrated into devices, i.e., `Things`. The gateway component mediates between Things and `Message Broker` components, although the AWS IoT architecture does not describe it. The IoT Integration Middleware component contains the main platform functionality and is composed of Message Broker, `Thing Shadows`, `Thing Registry`, `Rules Engine`, and the `Security & Identity` components. The application component on the right-hand side figure contains data processing services, such as `AWS Lambda` or `Amazon S3`. The application component on the bottom-hand side allows the connection to other IoT applications.

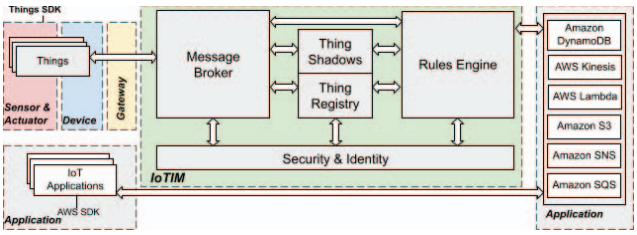

Fig. 3: AWS architecture for IoT

RAs can be particularly helpful since they capture the logical components of an architecture, abstracting from the implementation and the underlying technologies [19], [20], thus providing another abstraction level. The RA in Fig. 1 can be exploited to more easily interpret and compare the corresponding concrete architectures shown in Fig. 2 and Fig. 3. Indeed, using RAs is strongly recommended in some domains [7], [8], [23]. In the next section, we propose an approach based on mining and assistive modeling using RAs.

III. Assistive RA modeling

Assistive Modeling Tools [24] are tools usually integrated with ML techniques that can help engineers compose models in a meaningful way. In MDE, multiple tasks might be subject to recommendations, and numerous tools have been presented as modeling assistants [25]–[27]. These tools are generally inspired by Recommender Systems (RSs) used in software development [28]. For instance, RSs may help in choosing appropriate third-party programming libraries or recommend APIs. There is, however, a lack of support for RSs for architectural modeling, which are mainly based on RAs.

Our vision is that *effective RA modeling must be guided by a modeling assistant that provides interactive recommendations to support the instantiation, refinement, evolution, and migration of architectures with visualization and suggestions, continuously considering compliance with the RA*. In this way, the software architect is supported not only in the definition of new architectural models, but also in their evolution while respecting the chosen RA. The specific approach that we envision, namely AssistRA is sketched in Fig. 4. We rely on a modeling framework in charge of supporting all the modeling activities related to RAs and concrete architectures, for instance, the tool proposed in [29] or [30], [31] as ADL

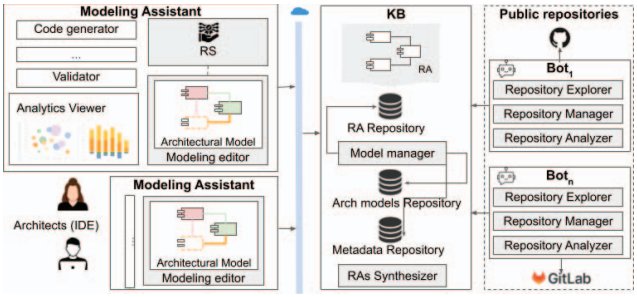

Fig. 4: The proposed AssistRA Framework

used for both the architecture and the RA. This is a prerequisite to support syntactic recommendations based on the typing relationship between the RA and the modeled architecture [32]. A Modeling editor, part of the Modeling Assistant, supports the modeling phase of both the software architectures and RAs. Both graphical [33] or textual [34] modeling can be supported. The resulting models for both of them can be stored in modeling repositories, locally or on the cloud [35], [36]. If the architect decides to be supported with a cloud-based solution, the architectural models can be stored in a Arch model repository that will enrich a complete **Knowledge-base** (KB). When the architect composes an architectural model, at all stages, suggestions for the addition or removal of components, not only in case of violations but also considering existing architectural models defined by (other) architects, will be provided. In some cases, edits or even more complex patterns can also be suggested to reach compliance. For instance, in Fig. 4, a violation is identified in the modeled architecture with respect to a chosen RA in the cloud-based KB. Indeed, a component or a connector in red is identified as a violation in the RA declaration, and redirection is then suggested (in orange) to reach the minimal conformance w.r.t. the RA. In the architectural model, the dotted elements are not part of the partial model but are generated based on suggestions of the recommender engine.

It is worth noting that the reported example is a simple one, but RA can be very complex and articulated, and for this reason, AssistRA can be of crucial relevance. The recommendations that can be provided are not limited to actions to reach compliance because RAs present a high degree of variability [37], and for this reason, multiple options can be considered by the architect. Due to this nature, we foresee consistent advancements provided by AssistRA. The architectural models could also be used for code generation (see component Code Generator) to enforce the correctness and quality of the generated system. Model-based testing and validation techniques can be used to improve the quality of the architectural model further; for this reason, we have also included a Validator component.

The modeling assistants act as the front-end of the approach, but we have already anticipated that the main problem with adopting RAs is their lack of availability due to their cost of specification and non-standard format of specification. For

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on September 22,2025 at 12:48:27 UTC from IEEE Xplore. Restrictions apply.

this reason, we have also included a BOT component that will help in different settings. A component called REPOSITORY EXPLORER is in charge of exploring public repositories and looking for specific artifacts representing architectural models. For instance, if we consider an AADL [30] architectural model defined through the OSATE IDE [34], the related format used to encode it is $.aadl2$. By exploiting the public open APIs of the existing software repositories, e.g., Github, this component will mine the repository and import the publicly available architectural models in the KB. Mining repositories is a mature field in software engineering [38] that aims at extracting information from software artifacts to capture and possibly derive hidden information on a large scale. In order to successfully extract information from architectural models, the analysis must be performed simultaneously on different repositories to combine the information obtained, and the more the population is extended, the more the process will be beneficial. The mining process could derive structural information, e.g., how the architecture is composed, or quality aspects assessed with specific metrics to be computed; also, metadata could be extracted from the repositories subject of the mining process. The Bot is also coupled with a MANAGER and an ANALYZER that will retrieve additional metadata from the repositories and will propagate the artifacts and metadata to the KB. The KB includes a crucial component, i.e., the RAS SYNTHESIZER, that is in charge of analyzing the dataset of the collected architectural models and generating candidate RAs. Heuristics will be included and encoded in the synthesizer, which will analyze the KB and use matching algorithms and similarity measures to build a cluster-based representation of the mega-repository constructed over time. For instance, given more architectures like the ones reported in Fig. 2 and 3, the synthesizer may extract a common RA for IoT and refine it continuously over time. To synthesize RAs, clustering techniques [39] might be helpful for grouping architectural models based on structural and content-based similarity, for instance resulting in domain-specific or architectural patterns-based or styles clusters. Once the most representative architecture (e.g., centroid) is selected, a generalization step might extract a flexible architecture corresponding to the lowest common denominator of the cluster.

Use cases. In the following, we list a set of use cases used to demonstrate the usefulness of the proposed approach.

Instantiation from scratch: when the architect needs to define her software architecture, ASSISTRA can suggest the minimal architecture structure to comply with the chosen RA based on the architectural notions and guidelines computed in the KB. The mined architectural models are crucial for the effectiveness of the suggestions.

Architectural Refinement for compliance: when the architect has already specified a partial architecture, and while selecting a RA, the result is not compliant, ASSISTRA can suggest the modeling actions to reach minimal compliance [29].

Architectural Completion: in the case when the compliance is reached, or it should be reached, other architectural models can be used to compute possible suggestions of the most used components and connectors for that RA. For instance, if the RA declares an access controller as an intermediary component for user access management, and most of the architects implemented it with specific components connected to data layers, ASSISTRA can use this information to generate suggestions, even if there are many ways to reach or to maintain the compliance. Also, here it is crucial to have a large dataset of architectural models obtained from repositories to foster the adoption of qualitative RAs.

RA matching: when the architect specifies a partial architecture by having general knowledge of the domain and architectural rules, ASSISTRA can suggest a RA (or more) compliant with the partial architecture (or the complete one). The architect can apply further refinements, and the search space of the matched RAs can be reduced during the process. When the architect selects the most relevant and fitting RA for her purpose, ASSISTRA can continue to work in refinement mode.

Technological Switch and Evolution: When an architectural model complies with a RA, the chosen RA assures that the architect built a system that can be deployed or released in compliance with the rules and guidelines dictated by third-party technological partners. A technological switch could be switching from one chosen RA to another. For instance, if the architecture complies with an IoT RA declared by AWS, and the architect wants to migrate to an Azure IoT RA, ASSISTRA can offer recommendations to reach compliance, if possible. Another interesting case is presented when the RA evolves [40], and the already defined concrete architectures must be migrated in case of broken compliance.

RA extraction: When software architects work with internal guidelines and regulations, they follow implicit RAs specifications. Given multiple declared architectures, ASSISTRA can try (if possible) to extract a common RA based on the declared architectural models. The KB will be continuously enriched with architectural specifications discovered by the Bots and analyzed with mining techniques to extract architectural knowledge. This continuous process will support ongoing updates of the KB for refinement of the RA extraction process.

IV. CHALLENGES AND FUTURE DIRECTIONS

In this paper, we envisioned a framework that helps software architects in defining their own architectures, in line with formalized RAs that can be derived by continuous knowledge refinement in the cloud. The knowledge refinement is based on a mining-in-the-wild approach that mines and discovers software repositories, importing architectural models and analyzing them to derive RAs. This methodology is enabled by the natural definition of RA, which is a template-based generalization of architectural solutions. Constructing a KB for architectures and reference architectures will enable the interaction with modeling assistants, and using recommender systems functionalities can support the architecting phase with specific actions to be undertaken to make the architecture compliant and improve the overall quality.

However, we identified the following key challenges to achieving this vision without pretending to be complete: i)

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on September 22,2025 at 12:48:27 UTC from IEEE Xplore. Restrictions apply.

reducing the burden of adopting RAs in the architecture lifecycle with modeling assistants that can significantly simplify the work of architects; ii) building a KB that is consistent and qualitative, excluding non-representative examples of architectural models; iii) feeding the modeling assistant with mining results that are truly helpful for software architects; iv) supporting the reuse, evolution, refactoring, and quality improvement of the architectural ecosystem.

Future work to address this ambitious vision will concern not only with the above challenges, but also with running experiments with software architects to study whether the modeling assistants help reduce the burden of defining software architectures and with running mining experiments to study how to extract RAs from existing repositories automatically.

Acknowledgments This work has been partially supported by the MUR under the grant EMELIOT - Engineered MachinE Learning-intensive IoT systems (PRIN 2020 program, Contract 2020W3A5FY) and the grant Regains "Dipartimenti di Eccellenza 2023-2027"

REFERENCES

[1] O. G. global consortium, "The open group," 2023. [Online]. Available: http://www.opengroup.org

[2] M. Weyrich and C. Ebert, "Reference architectures for the internet of things," *IEEE software*, vol. 33, no. 1, pp. 112–116, 2015.

[3] M. Haft, B. Humm, and J. Siedersleben, "The architect's dilemma–will reference architectures help?" in *1st Int. Conf. on the Quality of Software Architectures, and 2nd Int. Workshop on Software Quality, Erfurt, Germany, September 20-22*. Springer, 2005, pp. 106–122.

[4] M. Guessi, L. B. R. de Oliveira, and E. Y. Nakagawa, "Representation of reference architectures: A systematic review." in *SEKE*, 2011, pp. 782–785.

[5] S. Behere and M. Törngren, "A functional reference architecture for autonomous driving," *Inf. Softw. Technol.*, vol. 73, no. C, pp. 136–150, May 2016.

[6] A. Magnusson, L. Laine, and J. Lindberg, "Rethink ee architecture in automotive to facilitate automation, connectivity, and electro mobility," in *40th Int. Conference on Software Engineering: Software Engineering in Practice*. ACM, 2018, pp. 65–74.

[7] J. L. Tokar, "A comparison of avionics open system architectures," *ACM SIGAda Ada Letters*, vol. 36, no. 2, pp. 22–26, 2017.

[8] N. Hochgeschwender, G. Biggs, and H. Voos, "A reference architecture for deploying component-based robot software and comparison with existing tools," in *Second IEEE International Conference on Robotic Computing (IRC)*. IEEE, 2018, pp. 121–128.

[9] A. Grosskurth and M. W. Godfrey, "A reference architecture for web browsers," in *21st IEEE International Conference on Software Maintenance (ICSM'05)*. IEEE, 2005, pp. 661–664.

[10] Amazon, "Aws architecture center," 2023. [Online]. Available: https://aws.amazon.com/architecture/?nc1=h_ls

[11] Microsoft, "Azure architectures," 2023. [Online]. Available: https://docs.microsoft.com/en-us/azure/architecture/browse/

[12] IBM, "Ibm architecture center," 2023. [Online]. Available: https://www.ibm.com/cloud/architecture

[13] S. Angelov, P. Grefen, and D. Greefhorst, "A classification of software reference architectures: Analyzing their success and effectiveness," in *Joint Working IEEE/IFIP Conference on Software Architecture & European Conference on Software Architecture*. IEEE, 2009.

[14] L. Bass, P. Clements, and R. Kazman, "Software architecture in practice (third edition)," 2013.

[15] P. Kruchten, *The rational unified process: an introduction*. Addison-Wesley Professional, 2004.

[16] A. Bucaioni and P. Pelliccione, "Technical architectures for automotive systems," in *IEEE International Conference on Software Architecture (ICSA)*, 2020, pp. 46–57.

[17] P. Pelliccione, E. Knauss, S. M. Ågren, R. Heldal, C. Bergenhem, A. Vinel, and O. Brunnegård, "Beyond connected cars: A systems of systems perspective," *Science of Computer Programming*, vol. 191, pp. pp. 102–414, 2020.

[18] M. Bauer, M. Boussard, N. Bui, J. De Loof, C. Magerkurth, S. Meissner, A. Nettsträter, J. Stefa, M. Thoma, and J. W. Walewski, "Iot reference architecture," *enabling things to talk: designing IoT solutions with the IoT architectural reference model*, pp. 163–211, 2013.

[19] J. Guth, U. Breitenbücher, M. Falkenthal, P. Fremantle, O. Kopp, F. Leymann, and L. Reinfurt, *A Detailed Analysis of IoT Platform Architectures: Concepts, Similarities, and Differences*. Springer, 2018, pp. 81–101.

[20] J. Guth, U. Breitenbücher, M. Falkenthal, F. Leymann, and L. Reinfurt, "Comparison of iot platform architectures: A field study based on a reference architecture," in *Cloudification of the Internet of Things*. IEEE, 2016, pp. 1–6.

[21] F. Foundation, "Fiware," 2023. [Online]. Available: https://www.fiware.org/

[22] A. IoT, "Aws for internet of things (iot)," 2023. [Online]. Available: https://aws.amazon.com/iot/

[23] U. Eklund, Ö. Askerdal, J. Granholm, A. Alminger, and J. Axelsson, "Experience of introducing reference architectures in the development of automotive electronic systems," *ACM SIGSOFT Software Engineering Notes*, vol. 30, no. 4, pp. 1–6, 2005.

[24] E. Laurenzi, K. Hinkelmann, S. Jüngling, D. Montecchiari, C. Pande, and A. Martin, "Towards an assistive and pattern learning-driven process modeling approach." in *AAAI Spring Symposium: Combining Machine Learning with Knowledge Engineering*, 2019, p. 6.

[25] J. D. Rocco, C. D. Sipio, D. D. Ruscio, and P. T. Nguyen, "A gnn-based recommender system to assist the specification of metamodels and models," in *ACM/IEEE 24th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 2021.

[26] L. Almonte, E. Guerra, I. Cantador, and J. De Lara, "Recommender systems in model-driven engineering," *Software and Systems Modeling*, vol. 21, no. 1, pp. 249–280, 2022.

[27] S. Kögel, "Recommender system for model driven software development," in *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering*, 2017, pp. 1026–1029.

[28] P. Resnick and H. R. Varian, "Recommender systems," *Communications of the ACM*, vol. 40, no. 3, pp. 56–58, 1997.

[29] A. Bucaioni, A. D. Salle, L. Iovino, I. Malavolta, and P. Pelliccione, "Reference architectures modelling and compliance checking," *Softw. Syst. Model.*, vol. 22, no. 3, pp. 891–917, 2023.

[30] P. H. Feiler, D. P. Gluch, and J. J.Hudak, "The architecture analysis & design language (AADL): An introduction," Tech. Rep., (2006).

[31] Palladio, "Palladio: The software architecture simulator," 2022. [Online]. Available: https://www.palladio-simulator.com/home/

[32] V. V. G. Neto, L. Garces, M. Guessi, L. B. R. de Oliveira, and F. Oquendo, "On the equivalence between reference architectures and metamodels," in *1st International Workshop on Exploring Component-based Techniques for Constructing Reference Architectures (CobRA)*. IEEE, 2015, pp. 1–4.

[33] E. Kalnina, "Concrete syntax-based find for graphical dsls," in *23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 2020, pp. 1–5.

[34] C. M. University, "Osate (open source aadl tool environment)," 2023. [Online]. Available: https://osate.org/

[35] F. Basciani, J. Di Rocco, D. Di Ruscio, L. Iovino, and A. Pierantonio, "Model repositories: Will they become reality?" in *CloudMDE@MoDELS*, 2015, pp. 37–42.

[36] R. France, J. Bieman, and B. H. Cheng, "Repository for model driven development (remodd)," in *International Conference on Model Driven Engineering Languages and Systems*. Springer, 2006, pp. 311–317.

[37] E. Y. Nakagawa, "Reference architectures and variability: current status and future perspectives," *Proceedings of the WICSA/ECSA 2012 Companion Volume*, pp. 159–162, 2012.

[38] W. Poncin, A. Serebrenik, and M. Van Den Brand, "Process mining software repositories," in *15th European conference on software maintenance and reengineering*. IEEE, 2011, pp. 5–14.

[39] A. Ferdjoukh, F. Galinier, E. Bourreau, A. Chateau, and C. Nebut, "Measuring differences to compare sets of models and improve diversity in mde," in *ICSEA: International Conference on Software Engineering Advances*, 2017.

[40] R. Cloutier, G. Muller, D. Verma, R. Nilchiani, E. Hole, and M. Bone, "The concept of reference architectures," *Systems Engineering*, vol. 13, no. 1, pp. 14–27, 2010. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/sys.20129

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on September 22,2025 at 12:48:27 UTC from IEEE Xplore. Restrictions apply.